LiteTest Runner Internal Guide

Document Version: 1.0

Runner Version: 2026-06-11

Test Version: 1.0.0

This guide documents the internal architecture and design of the LiteTest Runner API and framework. It is intended for maintainers and contributors working on the implementation. It is companion document to the LiteTest Runner Internals Reference document.

Copyright (c) 2026 Paul Sinclair

License Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Preface

This document is intended for contributors.

For a list of other LiteTest documents and the LiteTest repository layout, see the LiteTest Documentation Guide.

For a glossary of terms, see the LiteTest Glossary Reference.

Document Version History

Document	Date	Runner	Test	Comment	Author/Editor
1.0	2026-06-11	1.0.0	1.0.0	Initial version.	Paul Sinclair

- The `Document` column records the document's version and uses the version format `M.u` (Major, update).
- The `Runner` column records the LiteTest Runner API version current at the time this document version was published and is defined by its `LT_RUNNER_VERSION` macro.
- The `Test` column records the LiteTest Test API version current at the time this document version was published and is defined by its `LT_TEST_VERSION` macro.

Both Runner and Test versions specify a string of the form `"M.m.p"` (Major, minor, patch). `M` is the same for the document, Runner, and Test.

The document's `u` (update) version track updates to this document and does not correspond to a `m` (minor) or `p` (patch) version. `u` increments whenever this document is updated without a change to `M`, and it resets to `0` when `M` is incremented.

Table of Contents

1. Introduction

Basic concepts and high-level description of the framework internals and design goals.

In this document, a *symbol* refers to any named entity in the LiteTest framework, including:

- typedefs
- structs
- enums and enum values
- macros
- variables
- functions

LiteTest exposes some internal symbols in `litetest_runner.h` because the framework's macros expand into code that depends on internal types and helper functions. These symbols must be visible to user code for the macros to compile correctly, but they are not part of the public API and should not be used directly.

Their presence in the header reflects C implementation requirements, not intended usage. Contributors may modify these symbols as needed, provided the public API contract remains intact.

1.1 Public and Internal Naming Conventions

Any framework names that are public and visible to LiteTest API users are prefixed with `lt_...` (typically lowercase) or `LT_...` (typically uppercase).

Any framework names that are internal (but technically visible to LiteTest API users) follow the pattern `litetest_..._internal_t`, `litetest_..._internal`, or `LITETEST_..._INTERNAL`. These names should not be referenced by API users.

In general, users of the API should not define names prefixed with `lt_`, `LT`, `litetest_`, or `LITETEST`, or reference names prefixed with `litetest_` or `LITETEST_`.

2. Architecture Overview

- Process model
- Thread model
- Signal handling
- Fault detection
- Execution flow

3. Orchestrator Internals

- Initialization
- Argument parsing
- Report lifecycle
- Category aggregation

---

4. Test Group Internals

- Group initialization
- Execution scheduling
- Result accumulation

5. Test Execution Engine

- Expression evaluation
- Isolation modes
- Concurrency model
- Error and fault propagation

6. Process-Isolated Execution

- Child process creation
- Monitoring and timeouts
- Exit code interpretation
- Fault mapping

7. Thread-Isolated Execution

- Thread creation
- Synchronization
- Fault boundaries

---

8. Signal Guard System

- Installed handlers
- Supported signals
- Fault classification
- Recovery behavior

---

9. Report Generation Internals

- Output formatting
- Category totals
- Fault messages
- Notes and metadata

10. Internal State and Global Variables

(Placeholder for internal state descriptions.)

11. Error Handling and Safety Guarantees

(Placeholder for internal error semantics.)

12. Implementation Notes

- Portability considerations
- POSIX dependencies
- Platform differences

13. Future Improvements

(Placeholder for roadmap items.)

Repository Layout

GitHub repository: `paulsinclair51/LiteTest`

Repository layout (listing core files):

```
LiteTest/
|- .github/
|  \- workflows/
|     \- ci.yml
|- README.md
|- LICENSE
|- Makefile
|- build_test_litetest.ps1
|- build/
|- docs/
|  \- LiteTest_API_User_Guide.md
|  \- LiteTest_API_Reference.md
|  \- LiteTest_Contributor_Guide.md
|  \- LiteTest_Framework_Guide.md
|  \- LiteTest_Framework_Reference.md
|- examples/
|- include/
|  \- litetest_runner.h
|- reports/
|  |- litetest_test_report-I.txt
|  \- litetest_test_report.txt
|- scripts/
|- src/
|  \- litetest_runner.c
|- tests/
|  |- test_litetest.c
|  |- test_orchestrator.c
|  |- test_guard1.c
|  \- test_guard2.c
```

Use this as a reference when adapting LiteTest into your own project structure.

Glossary

For general LiteTest terms, see the LiteTest Glossary document.

Runner-Specific Terms:

- **Orchestrator Lifecycle**: The sequence of initialization, group execution, test execution, and report finalization performed by the LiteTest framework.
- **Guard Behavior**: The mechanism LiteTest uses to catch runtime faults (e.g., segmentation faults) and continue executing remaining tests.
- **Isolation Semantics**: The rules governing how tests and groups run in threads or processes to prevent interference and ensure fault containment.
- **Concurrency Model**: The framework's rules for running tests concurrently within `LT_BEGIN_CONCURRENT` / `LT_END_CONCURRENT` blocks.
- **Execution Phases**: The internal stages of orchestrator operation, including initialization, argument parsing, group dispatch, test dispatch, and report writing.
- **Fault Handling Model**: The framework's strategy for capturing and reporting faults without terminating the entire test run.
- **Control File Promotion**: The process of replacing an outdated control file with a newly generated file when differences are expected or intentional.
- **Nested Group Behavior**: The rules governing how groups may contain other groups and how isolation and concurrency propagate through nested structures.
- **Concurrent Block Behavior**: The semantics of executing multiple tests in parallel within a concurrent block, including ordering and isolation rules.